Experiment No 9

Experiment Title: MPI Program 1

Problem Statement: Write MPI Program to print "Hello World".

MPI program to send and receive Hello World messages from all other processes to a Root process and print the received messages.

Objectives

1. **Understand MPI Basics**: Gain a basic understanding of MPI (Message Passing Interface) and its use in parallel programming.
2. **Learn MPI Communication**: Familiarize yourself with MPI functions for sending and receiving messages between processes.
3. **Implement a Simple MPI Program**: Write and execute a simple MPI program to print "Hello World" and send/receive messages between processes.

Theory

MPI (Message Passing Interface) is a standardized and portable message-passing system designed to function on a wide variety of parallel computing architectures. MPI provides a way for processes to communicate with one another by sending and receiving messages.

Key MPI Functions:

1. **MPI_Init**: Initializes the MPI environment.
2. **MPI_Comm_size**: Determines the size of the communicator (number of processes).
3. **MPI_Comm_rank**: Determines the rank of the calling process within the communicator.
4. **MPI_Send**: Sends a message to a specified process.
5. **MPI_Recv**: Receives a message from a specified process.
6. **MPI_Finalize**: Terminates the MPI environment.

Conclusion

This program demonstrates the basics of MPI, including initializing the environment, retrieving process information, and performing message passing between processes. By printing "Hello World" from each process and sending/receiving messages to/from the root process, you can see how MPI enables parallel programs to communicate effectively. Understanding MPI's fundamental functions is essential for developing more complex parallel applications that require efficient inter-process communication.

Source Code and Output /Screenshots:

(To be provided by the student)

//Source code and output/screenshot should be available here

mit90@login02:~

```
#include <mpi.h>
#include <stdio.h>
#include <string.h>

int main(int argc, char* argv[])

{
        int rank, size;
        MPI_Status status;

        char message[100];
        char recv_msg[100];

        MPI_Init(&argc, &argv);
        MPI_Comm_rank(MPI_COMM_WORLD, &rank);
        MPI_Comm_size(MPI_COMM_WORLD, &size);


        sprintf(message, "Hello World from process %d of %d", rank,size);

        if (rank == 0)
        {
                printf("%s\n", message);

                for (int i=1; i<size; i++)
                {
                        MPI_Recv(recv_msg, 100, MPI_CHAR, i, 0, MPI_COMM_WORLD, &status);
                        printf("%s\n", recv_msg);
                }
        }
        else
        {
                MPI_Send(message, strlen(message)+1, MPI_CHAR, 0, 0, MPI_COMM_WORLD);
        }

        MPI_Finalize();
        return 0;

}
"hello_mpi.c" 39L, 656C
```

```
[mit90@login02 ~]$ vi hello_mpi.c
[mit90@login02 ~]$ mpicc hello_mpi.c -o hello_mpi
[mit90@login02 ~]$ mpirun -np 4 ./hello_mpi
Hello World from process 0 of 4
Hello World from process 1 of 4
Hello World from process 2 of 4
Hello World from process 3 of 4
[mit90@login02 ~]$
```